HACKBRIDGE

Intelligent Inter-College Hackathon Collaboration Platform

Development Workflow and Work Division for a 4-Member Team

The project should be divided by **feature ownership** rather than separating frontend, backend, and database completely. Each member owns complete modules - UI, API integration, logic, and testing - while the database lead maintains the common schema.

1. Main Development Modules

#	Module	Main Functionality
1	Authentication & User Profiles	Registration/login, JWT, bcrypt, role-based access, student/mentor/college/admin profiles
2	Team Management & Smart Formation	Create/join teams, applications, auto team formation, teammate recommendations, completeness score
3	Hackathon & Announcement Management	Hackathon CRUD, organizer announcements, deadlines, feeds and hackathon details
4	Eligibility & Hackathon Recommendation	Eligibility engine, weighted match %, explanations and recommendations
5	Mentor Management & Matching	Mentor profiles, search, mentorship requests and mentor match scoring
6	Search & Notifications	Advanced filters, invitations, application updates, deadline and recommendation notifications
7	DDBMS & Database Architecture	MySQL schema, fragmentation, replication, distributed queries and inter-college access
8	Dashboards, Analytics & Administration	Role dashboards, charts, platform statistics and admin controls

2. Division Between 4 Members

Member	Primary Responsibility	Modules
Person 1	Authentication + Profiles + Database/DDBMS Lead	1 & 7
Person 2	Team Formation + Search + Notifications	2 & 6
Person 3	Hackathons + Eligibility + Recommendation Engine	3 & 4
Person 4	Mentor Matching + Dashboards + Admin	5 & 8

3. Detailed Responsibility of Each Member

Person 1 - Authentication, Profiles & DDBMS

• Register/login for Student, Mentor, College/Organizer and Admin

• JWT authentication and bcrypt password hashing

• Role-based authorization and profile management

• Master MySQL schema, primary/foreign keys, indexes and normalization

• Simulated college databases/schemas for DDBMS

• Distributed queries, fragmentation, replication and location transparency

• Maintain the master database design requested by all other members

Person 2 - Teams, Smart Formation, Search & Notifications

• Create team, join/apply, accept/reject applications and team requirements

• Implement 'I Don't Have a Team' automatic formation pool

• Complementary role/skill matching instead of grouping similar students

• Team completeness percentage and missing-role detection

• Recommend students for missing team positions

• Advanced student/team/mentor search and filters

• Notifications for team invitations, applications, missing roles and updates

Person 3 - Hackathons, Eligibility & Recommendation Engine

• Hackathon CRUD and organizer announcement publishing

• Hackathon feed, registration deadlines and detailed information

• Eligibility checks for branch, year, college, location and team size

• Explain why a student/team is not eligible

• Weighted Team-Hackathon Match Score

• Explain score using skills, domain, project relevance, experience and completeness

• Track hackathon applications and recommendation results

Person 4 - Mentors, Dashboards & Administration

• Mentor profiles, expertise, technologies, domains and availability

• Mentorship request, accept and reject workflow

• Weighted Mentor Match Score with explanation

• Student, Mentor, College and Admin dashboards

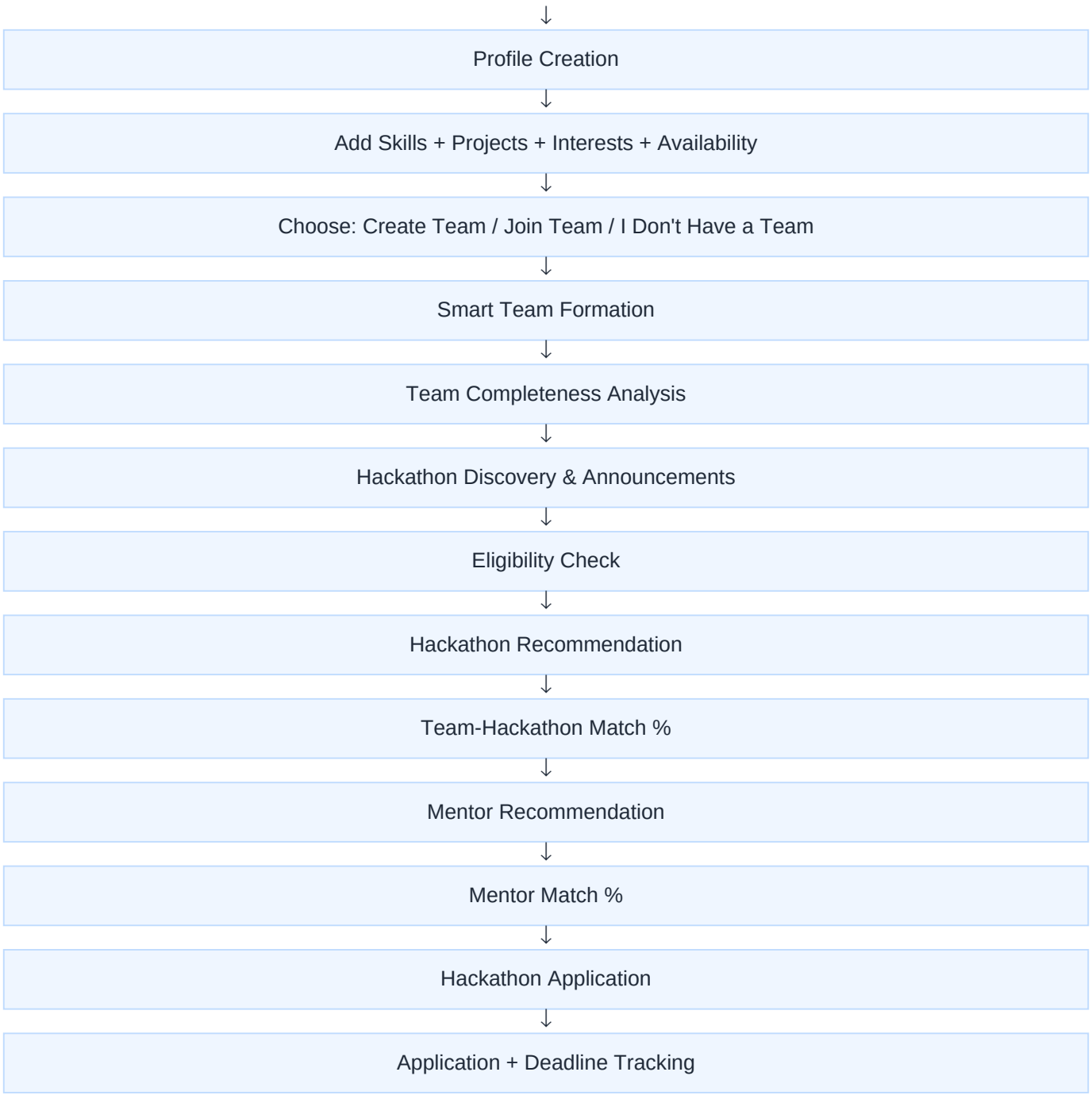
• Charts and platform analytics

• Admin management of users, colleges, mentors, organizers and hackathons

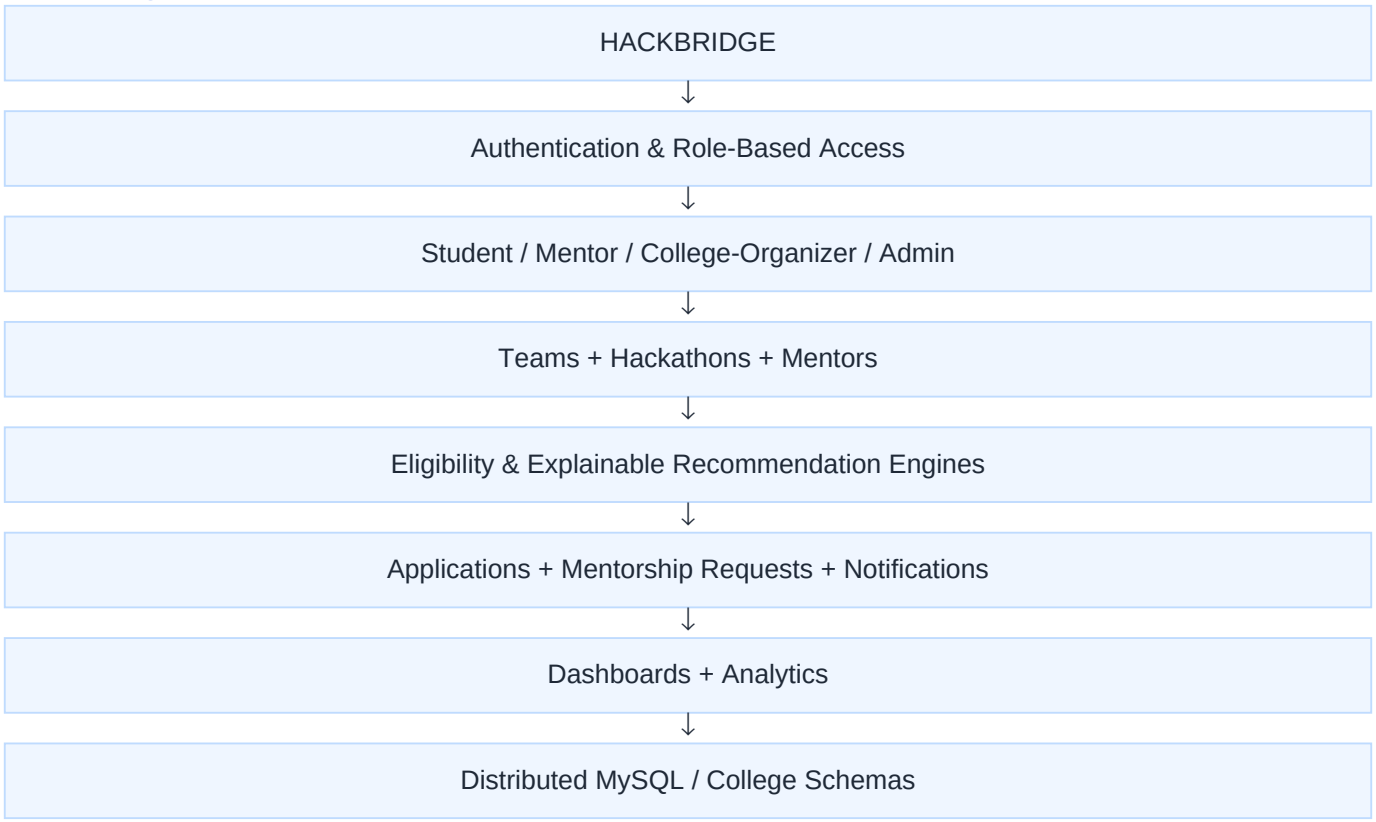
• Organizer verification and moderation functionality

4. Core Application Workflow

Student Registration / Login


Profile Creation
Add Skills + Projects + Interests + Availability
Choose: Create Team / Join Team / I Don't Have a Team
Smart Team Formation
Team Completeness Analysis
Hackathon Discovery & Announcements
Eligibility Check
Hackathon Recommendation
Team-Hackathon Match %
Mentor Recommendation
Mentor Match %
Hackathon Application
Application + Deadline Tracking

5. Integration Architecture



6. Recommended Development Order

Phase 1: Git/GitHub setup -> React + Express structure -> master database schema -> authentication.

Phase 2: Profiles -> Teams -> Hackathons -> Mentor profiles.

Phase 3: Team applications -> announcements -> mentorship requests -> search.

Phase 4: Team completeness -> automatic team formation -> eligibility -> hackathon match -> mentor match.

Phase 5: Notifications -> dashboards -> analytics -> admin.

Phase 6: DDBMS simulation -> distributed queries -> integration testing -> deployment.

7. Important Team Rule

All members can develop the database requirements for their modules, but **Person 1 should maintain the master schema**. This avoids duplicate tables, inconsistent foreign keys and integration problems. Agree on common REST API naming before development.

Suggested API structure: /api/auth, /api/students, /api/teams, /api/hackathons, /api/mentors, /api/notifications, /api/admin

Key Demonstration Features

1. Smart Team Formation - forms balanced teams using complementary skills.
2. Explainable Hackathon Matching - calculates eligibility and Team-Hackathon Match % with reasons.
3. Distributed Inter-College Search - demonstrates DDBMS concepts across simulated college databases.